

字符串理论基础

String

黄万平

2026 年 7 月 16 日



① Formal Notation

② String Hash

③ KMP Algorithm

④ Border Theory

⑤ Trie

① Formal Notation

② String Hash

③ KMP Algorithm

④ Border Theory

⑤ Trie

前言

课件用了四年了，不妨再用一年。

课件用了五年了，不妨再用一年。

第一版课件作者：z**bh**

第二版课件修改者：w**zk**

第三版课件修改者：c**yy**

第四版课件修改者：w**yh**

第五版课件修改者：d**xj**

图 1: 无敌了.

第六年，重做了一遍.

第七年，沿用了去年课件.

前言

字符串理论基础. 主讲人: 黄万平.

若有任何疑问, 欢迎大家于:

QQ: 2586647166

邮箱: hwpvector@foxmail.com

联系我.

上课过程中, 有任何地方有疑问, 请即时打断.

由于课程难度、时长控制, 无法讲述 manacher 和 Z 函数算法.
有志于进一步提升的同学, 建议寻找优质博客自学.

字符

字符集 Σ : 组成字符串元素的集合. 如:

- $\Sigma = \{a, b, c, \dots, z, A, B, C, \dots, Z\}$.
- $\Sigma = \{0, 1, 2, \dots, 9\}$.
- $\Sigma = \{x \in \mathbb{N} \mid x \leq n\}$.

字符 c : 字符集 Σ 中的元素.

为了比较字符大小, 我们预先规定字符集 Σ 上的一个严格全序关系 $<$. 我们称该全序关系为字符的字典序, 例如:

- $\Sigma = \{a, b, c, \dots, z\}$, 字典序可以为 $a < b < c < \dots < z$.
- $\Sigma = \{0, 1, 2, \dots, 9\}$, 字典序可以为 $0 < 1 < 2 < \dots < 9$.

由字符的字典序, 我们可以比较任意两个字符的大小.

字符串

字符串是若干字符按顺序排列得到的序列. 例如, 在字符集 $\Sigma = \{a, b, c, \dots, z\}$ 上, $S = \text{abacabad}$ 是一个字符串.

形式化地, 长度为 n 的字符串写作 $S = S[1]S[2] \cdots S[n]^1$, 其中每个 $S[i] \in \Sigma$.

上例中 $|S| = 8$, $S[1] = a$, $S[4] = c$, $S[8] = d$.

记空字符串为 ϵ . $|\epsilon| = 0$.

由字符集 Σ 组成的所有字符串构成的集合为 Σ^* .

记 $\Sigma^+ = \Sigma^* \setminus \{\epsilon\}$.

¹本课程字符串下标默认从 1 开始.

子串

在 $S = abacabad$ 中截取第 3 到第 6 个字符, 得到 $S[3,6] = acab$, 这是一段连续的字符, 称为 S 的子串.

子串必须连续. 例如, 按顺序选取第 1, 3, 5, 8 个字符得到 $aaad$, 它是子序列, 但不是子串.

形式化地, 字符串 S 的子串 $S[i,j]$ ($1 \leq i \leq j \leq |S|$) 为 $S[i]S[i+1] \cdots S[j]$.

特殊的, ϵ 是任意字符串的子串.² 通常, 记 $S[i+1,i] = \epsilon$ ($1 \leq i < |S|$)

本课程需要表示空子串时, 将其单独记为 ϵ .

²在一些题目中, 可能认为 ϵ 不是字符串的子串. 需要具体阅读题目条件. ☰

子串

对于字符串 S 的两个子串 $S[i_1, j_1]$ 和 $S[i_2, j_2]$, 记:

- 这两个子串不同, 当且仅当 $i_1 \neq i_2$ 或者 $j_1 \neq j_2$. 特殊的, 空串与其他任何非空子串不同.
- 对于任意一个字符串 s , 有 $\frac{|s|(|s|+1)}{2} + 1$ 个不同的子串.
- 这两个子串本质不同, 当且仅当 $s[i_1, j_1] \neq s[i_2, j_2]$. 特殊的, 空串与其他任何非空子串本质不同.

例如, 对于 $S = \text{ababa}$, 按位置区分的非空子串共有 15 个; 其中 $S[1, 3] = \text{aba}$ 与 $S[3, 5] = \text{aba}$ 是两个不同的子串, 但二者本质相同.

其本质不同的非空子串包括

$a, b, ab, ba, aba, bab, abab, baba, ababa$.

子序列

字符串 S 的一个子序列 S' 满足：存在 k 个正整数

$p_1, p_2, p_3, \dots, p_k$, 满足 $1 \leq p_1 < p_2 < p_3 \cdots < p_k \leq n$, 使得
 $S' = S[p_1]S[p_2]S[p_3] \dots S[p_k]$.

特殊的, ϵ 是任意字符串的子序列.

例如, 在 $S = \text{algorithm}$ 中选取位置 1, 3, 5, 7, 9, 得到子序列
 agrtm ; 由于这些字符在原串中不连续, 它不是子串.

前缀后缀

$S[1, k]$ ($0 \leq k \leq |S|$) 称为 S 的长度为 k 的前缀.

$S[|S| - k + 1, |S|]$ 称为 S 的长度为 k 的后缀.

特殊的, ϵ 是任意一个字符串的前缀和后缀.

记 $\text{pref}(S)$ 表示所有 S 的前缀的集合.

记 $\text{suf}(S)$ 表示所有 S 的后缀的集合.

不等于原串的前缀、后缀分别称为真前缀、真后缀.

例如, 对于字符串 $S = \text{abacaba}$, 有:

- $\text{pref}(S) = \{\epsilon, a, ab, aba, abac, abaca, abacab, abacaba\}$.
- $\text{suf}(S) = \{\epsilon, a, ba, aba, caba, acaba, bacaba, abacaba\}$.

拼接

记字符串 $S = S_1 S_2$ 表示 S_1 和 S_2 拼接的字符串，即：
 $S = S_1[1]S_1[2]S_1[3] \dots S_1[|S_1|]S_2[1]S_2[2]S_2[3] \dots S_2[|S_2|]$.

记字符串的零阶幂 $S^0 = \epsilon$. 递归定义字符串的 n 阶幂为
 $S^n = S^{n-1}S$.

例如, $(ab)^3c^2 = abababcc$.

字典序

比较两个字符串时，从左到右找到第一个不同的位置：

- 若存在不同位置，则该位置字符较小的字符串更小。
- 否则，若一个字符串是另一个字符串的前缀，则较短的字符串更小。
- 否则，两字符串相同。

< 在字符串集合 Σ^* 上也是一个严格全序关系。由字符串的字典序，我们可以比较任意两个字符串的大小。

例如：abac < abaca < abacb < abb < bac.

① Formal Notation

② String Hash

③ KMP Algorithm

④ Border Theory

⑤ Trie

字符串哈希简介

将字符串映射到一个可以被比较的整数，被称为哈希值。该函数被称为哈希函数。具体地：

- 若两个哈希值不同，则两个字符串一定不同。
- 若两个哈希值相同，则两个字符串可能相同，也可能不同（此时我们称之为哈希碰撞）。

在竞赛中，哈希碰撞概率很低。常把哈希相等作为字符串相等的判断。

对长度为 n 的字符串进行 $O(n)$ 预处理后，可以 $O(1)$ 比较两个子串的哈希值，从而以很高概率判断它们是否相同。

字符串哈希

十进制数可以展开为: $1234 = 1 \cdot 10^3 + 2 \cdot 10^2 + 3 \cdot 10 + 4$.

类似地, 为每个字符分配一个不同的正整数 $f(c)$, 取基数 b , 并保证 $1 \leq f(c) < b$. 例如 $a \mapsto 1, b \mapsto 2, c \mapsto 3$, 则
 $abc \mapsto 1 \cdot b^2 + 2 \cdot b + 3$.

这个整数可能很大, 因此选取模数 $p > b$ (通常取质数), 只保留模 p 的余数:

$$H(S) = \left(\sum_{i=1}^n f(S[i]) b^{n-i} \right) \bmod p.$$

上面这个函数, 就是哈希函数. 对应的值, 就是该字符串的哈希值.

字符串哈希

假设我们已经计算出一个前缀 $S[1, i]$ 的哈希值. 则:

$$H(S[1, i + 1]) = (H(S[1, i]) \cdot b + f(S[i + 1])) \bmod p$$

约定 $H(S[1, 0]) = 0$. 于是, 我们可以在 $O(n)$ 的复杂度给出一个字符串所有前缀的哈希值.

字符串哈希

记 $h[i] = H(S[1, i])$, $pw[k] = b^k \bmod p$, 并令 $h[0] = 0$.

对于任意一个子串 $S[l, r]$, 去掉前缀 $S[1, l-1]$ 对应的高位贡献, 有:

$$H(S[l, r]) = (h[r] - h[l-1] \cdot pw[r-l+1]) \bmod p.$$

于是, 在预处理所有前缀的哈希值、 b^k 在 $0 \leq k \leq |S|$ 时的值后, 我们可以 $O(1)$ 的给出任意一个子串的哈希值.

哈希碰撞

我们发现当哈希值相同的时候，两串不一定一样。这种现象被称为哈希碰撞。

在哈希值近似均匀分布的假设下，指定两个不同字符串发生碰撞的概率约为 $\frac{1}{p}$ 。

若共计算 q 个哈希值，并讨论其中是否存在任意一对碰撞，根据生日悖论，其风险大致随 $\frac{q^2}{2p}$ 增长。

当我们需要 $O(\sqrt{p})$ 个字符串是否完全相同，且得到的哈希值都相同的时候，此时有很大的概率，发生哈希碰撞。

哈希碰撞

为降低碰撞概率，我们可以对一个字符串选取 2 组 (b, p) ，分别计算哈希值。判断两组哈希值是否都相等，来判定两串是否相同。这种方式称为双哈希。

选取 1 组 (b, p) 的方式称为单哈希。

同理，我们有三哈希。

为降低碰撞概率，我们也可以取更大的 p 作为模数。

哈希碰撞

下面我们称当 p 在 10^{18} 数量级为大模数. 在 10^9 数量级称为小模数. 实际上, 根据相关研究, 在字符串长度 $|S| \leq 10^6$ 范围内, 我们有:

- 小模数单哈希在概率上可能出现问题. 极有可能发生碰撞. 尽量少使用.
- 大模数单哈希、小模数双哈希在概率上基本不会出现问题. 基本不会发生碰撞. 在 OI, IOI 和 ACM 赛制中使用.
- 当出题人已知你预设的所有 (b, p) , 可以通过一些攻击手段, 构造出两组发生哈希碰撞的字符串.³
- 双哈希使用随机基数、随机模数, 目前是无法被卡的. 单哈希使用随机基数、固定大模数, 双哈希使用随机基数、固定小模数, 可以提高取模效率, 同时也是稳妥的. 在 CF 赛制中使用.

³可以参考: <https://www.luogu.com.cn/article/fx07jjaz>              

示例代码

```
1  const long long b[2] = {13331, 19260817};
2  const long long p[2] = {998244353, 1000000007};
3  long long h[2][MAX_N], powb[2][MAX_N];
4  char s[MAX_N]; int n;
5  void init() {
6      for (int k = 0; k < 2; k++) {
7          h[k][0] = 0; powb[k][0] = 1;
8          for (int i = 1; i <= n; i++) {
9              h[k][i] = (h[k][i - 1] * b[k] + s[i])
10                  % p[k];
11              powb[k][i] = (powb[k][i - 1] * b[k])
12                  % p[k];
13          }
14      }
15 }
```

示例代码

```
1 long long get(int k, int l, int r) {
2     return (h[k][r] + p[k]
3         - powb[k][r - l + 1] * h[k][l - 1] % p[k])
4         % p[k];
5 }
6 bool isequal(int l1, int r1, int l2, int r2) {
7     return get(0, l1, r1) == get(0, l2, r2) &&
8         get(1, l1, r1) == get(1, l2, r2);
9 }
```

字符串哈希应用

下面列出若干可以通过字符串哈希算法解决的经典问题.

- 字符串匹配.
- 任意两个子串的最长公共前缀 (LCP) .
- 任意两个子串的字典序大小.
- 允许失配 k 次的字符串匹配.
- 最长回文子串.

字符串匹配

对于两个字符串 S 和 T . 在文本串 T 中寻找哪些子串与模式串 S 相同, 被称为字符串匹配.

我们求出 S 的哈希值后, 求出 T 所有长度为 $|S|$ 的子串的哈希值, 比较即可.

复杂度为 $O(|S| + |T|)$.

最长公共前缀

对于两个字符串 S 和 S' , 存在最大的非负整数 $0 \leq k \leq \min\{|S|, |S'|\}$, 满足 $S[1, k] = S'[1, k]$. 我们称 k 为二者的最长公共前缀长度 (LCP).

例如, $\text{LCP}(\text{abacaba}, \text{abacus}) = 4$; $\text{LCP}(\text{abc}, \text{xyz}) = 0$.

给定一个字符串 S , 现在需要询问其任意两个子串 S_1 和 S_2 的 LCP.

要求单次询问复杂度为 $O(\log \min\{|S_1|, |S_2|\})$, 预处理复杂度为 $O(|S|)$.

最长公共前缀

记 $k = \text{LCP}(S_1, S_2)$.

给定一个字符串 S , 现在需要询问其任意两个子串 S_1 和 S_2 的 LCP.

公共前缀有二分性. 若 $k_1 \leq k_2$, 且 $S_1[1, k_2] = S_2[1, k_2]$, 则有 $S_1[1, k_1] = S_2[1, k_1]$. 公共前缀的前缀仍然是公共前缀.

考虑二分 k 的大小. 通过哈希判断 $S_1[1, k]$ 和 $S_2[1, k]$ 是否相同, 判断 k 偏大还是偏小.

单次询问复杂度为 $O(\log \min\{|S_1|, |S_2|\})$.

字典序大小

给定一个字符串 S ，现在需要询问其任意两个子串 S_1 和 S_2 的字典序大小。

求出 $k = \text{LCP}(S_1, S_2)$ ，然后分情况判断：

- 若 $k = |S_1| = |S_2|$ ，则两串相等。
- 若 $k = |S_1|$ ，则 $S_1 < S_2$ ；若 $k = |S_2|$ ，则 $S_2 < S_1$ 。
- 否则，比较 $S_1[k+1]$ 与 $S_2[k+1]$ 。

单次询问复杂度为 $O(\log \min\{|S_1|, |S_2|\})$ 。

允许失配 k 次的字符串匹配

对于两个字符串 S 和 T . 在文本串 T 中寻找哪些子串与模式串 S 长度相等、且只有小于等于 k 个对应的字符不相同, 被称为允许失配 k 次的字符串匹配. 此处 k 较小.

要求时间复杂度为 $O(k|T|\log|S|)$.

允许失配 k 次的字符串匹配

在文本串 T 上枚举候选子串的起点 $1 \leq s \leq |T| - |S| + 1$; 只在窗口 $T[s, s + |S| - 1]$ 内比较.

例如 $S = \text{abcdefg}$, 候选窗口为 abxdexg , 允许失配 2 次:

$\text{ab} \mid \text{x} \mid \text{de} \mid \text{x} \mid \text{g}$
LCP $\rightarrow$ 跳过失配 $\rightarrow$ LCP $\rightarrow$ 跳过失配 $\rightarrow$ LCP

每次用哈希和二分求出当前两段的 LCP, 跳过首个失配字符后继续.

不断循环该过程, 直至 $|S|$ 被匹配完全, 或者已经失配 $k + 1$ 次.

复杂度为 $O(k|T|\log|S|)$.

最长回文子串

记字符串 S 的反串 $S^R = S[|S|]S[|S| - 1]S[|S| - 2] \cdots S[1]$.

一个字符串 S 被称为回文串, 当且仅当 $S = S^R$.

给定一个字符串 S , 现在需要询问最长的回文子串. 要求复杂度为 $O(|S| \log |S|)$.

最长回文子串

记字符串 S 的反串 $S^R = S[|S|]S[|S| - 1]S[|S| - 2] \cdots S[1]$.

一个字符串 S 被称为回文串, 当且仅当 $S = S^R$.

我们通过回文中心, 来枚举子串.

回文中心, 就是回文串的中心的位置, 具体地:

- 长度为奇数的回文串, 其回文中心位于某个字符 i , 例如 `racecar` 的中心是 `e`.
- 长度为偶数的回文串, 偶回文的中心位于相邻字符 i 与 $i + 1$ 之间, 例如 `abccba` 的中心位于两个 `c` 之间.

最长回文子串

给定一个字符串 S ，现在需要询问最长的回文子串。

分别枚举奇回文中心 i ，以及偶回文中心 $(i, i+1)$ 。

固定中心后，回文半径具有二分性：半径为 r 的子串是回文串，则更小半径对应的子串也一定是回文串。

因此二分回文半径，并通过正串与反串的子串哈希判断当前半径是否可行。

如何计算 $(S[l, r])^R$ 的哈希值？

由于 $(S[l, r])^R = S^R[|S| - r + 1, |S| - l + 1]$ ，预处理 S^R 的所有前缀的哈希值即可。

复杂度为 $O(|S| \log |S|)$ 。

① Formal Notation

② String Hash

③ KMP Algorithm

④ Border Theory

⑤ Trie

从字符串匹配说起

考虑在文本串 $T = \text{ababababac}$ 中匹配模式串 $S = \text{abababac}$.

第一次对齐时, 模式串的前 7 个字符均匹配成功, 但最后一个字符失配:

T: ababababac

S: abababac

^

若接下来发生失配, 朴素算法会将模式串向后移动一位, 从头重新比较.

我们总是想要下一个 T 的字符满足匹配, 我们能否不动当前 T 匹配到哪一个位置? 去改变 S 开头的位置, 让 T 的下一个字符满足匹配?

失配后可以保留什么？

对已经匹配的字符串 abababa, 有

ababa ba ab ababa.
 └───┘ └───┘
 前缀 后缀

因此前一次匹配的末尾 ababa, 可以直接作为下一次匹配的开
头；我们不必重新比较这 5 个字符.

T: ababababac
S: abababac

接下来只需比较最后三个字符 bac, 即可发现模式串匹配成功.

字符串匹配中, 失配后能够复用的信息, 正来自这种既是前缀、
又是后缀的字符串. 这样的字符串称为原串的 border.

Border 定义

若字符串 S 和 T 满足:

- $S \neq T$.
- S 是 T 的前缀.
- S 是 T 的后缀.

则称 S 是 T 的一个 border.

$\text{border}(S)$ 表示 S 的所有 border 构成的集合.

$\text{maxborder}(S)$ 表示 S 中长度最长的 border.

例如, 对于字符串 $S = aabaabaa$:

- $\text{border}(S) = \{aabaa, aa, a, \epsilon\}$.
- $\text{maxborder}(S) = aabaa$.

Border 性质

定理 1

对于任意字符串 S , 有:

$$\text{border}(S) = \{\text{maxborder}(S)\} \cup \text{border}(\text{maxborder}(S)).$$

即 maxborder 加上 maxborder 的 border 恰是原串的所有 border .

Border 性质证明

证明:

先证 $\{\text{maxborder}(S)\} \cup \text{border}(\text{maxborder}(S)) \subseteq \text{border}(S)$.

设 $\text{maxborder}(S) = S[1, m] = S[|S| - m + 1, |S|]$.

设 $S[1, m]$ 有 border $S[1, k] = S[m - k + 1, m]$.

由于 $S[m - k + 1, m]$ 是 $S[1, m]$ 的后缀, 故也是 $S[|S| - m + 1, |S|]$ 的后缀, 故有

$S[m - k + 1, m] = S[|S| - k + 1, |S|]$.

故有 $S[1, k] = S[|S| - k + 1, |S|]$. 得证.

即 border 的 border 还是原串的 border.

Border 性质证明

后证 $\text{border}(S) \subseteq \{\text{maxborder}(S)\} \cup \text{border}(\text{maxborder}(S))$.

设 $\text{maxborder}(S) = S[1, m] = S[|S| - m + 1, |S|]$.

对于任意的 $S[1, k] \in \text{border}(S)$, 有 $S[1, k] = S[|S| - k + 1, |S|]$.

$k \leq m$, 所以有 $S[1, k]$ 是 $S[1, m]$ 的前缀, $S[|S| - k + 1, |S|]$ 是 $S[|S| - m + 1, |S|]$ 的后缀.

故有 $S[1, k]$ 是 $S[1, m]$ 的后缀. 故 $S[1, k]$ 是 $S[1, m]$ 的 border.
得证.

前缀 Border

若我们知道一个字符串 S 所有前缀的 maxborder 的长度. 我们能否表示出其所有 border 的长度? 记 $S[1, i]$ 的 maxborder 的长度为 π_i 或者 $\text{next}[i]$ 或者 $\text{fail}[i]$. 下文中记为 $\text{fail}[i]$.

根据 $\text{border}(S) = \{\text{maxborder}(S)\} \cup \text{border}(\text{maxborder}(S))$, 我们有 $\text{border}(S) = \{\text{maxborder}(S), \text{maxborder}(\text{maxborder}(S))\} \cup \text{border}(\text{maxborder}(\text{maxborder}(S)))$. 不断代入, 有 $\text{border}(S) = \{\text{maxborder}(S), \text{maxborder}(\text{maxborder}(S)), \text{maxborder}(\text{maxborder}(\text{maxborder}(S))), \dots\}$.

何时停止迭代? 当一个串只存在空串是其 border 时停止.

即: $\text{border}(S) = \{S[1, \text{fail}[|S|]], S[1, \text{fail}[\text{fail}[|S|]]], S[1, \text{fail}[\text{fail}[\text{fail}[|S|]]]], \dots\}$.

前缀 Border

现在我们需要对一个字符串 S , 求 $fail[1], fail[2], \dots, fail[|S|]$.

考虑递推. 设我们已经求出了 $i < n$ 的所有 $fail[i]$. 现在需要求 $fail[n]$.

设 $x = fail[n]$. 则 $S[1, x] = S[n - x + 1, n]$, 则 $S[1, x - 1] = S[n - x + 1, n - 1]$.

同时 $x \neq n$, 故 $S[1, x - 1]$ 是 $S[1, n - 1]$ 的 border.

考虑从大到小枚举可行的 $x - 1$ 的值. 即枚举 $fail[n - 1]$, $fail[fail[n - 1]]$, $fail[fail[fail[n - 1]]]$, $\dots$, 直至枚举到 0 停止.

设枚举到的 border 长度为 j . 检查 $S[j + 1] = S[n]$ 是否成立.

若成立, 则 $fail[n] = j + 1$; 若一直不成立, 则 $fail[n] = 0$.

示例代码

```
1  int fail[MAX_N], n;  
2  char s[MAX_N];  
3  void getmaxborder() {  
4      fail[0] = fail[1] = 0;  
5      for (int i = 2, j = 0; i <= n; i++) {  
6          while (j > 0 && s[j + 1] != s[i])  
7              j = fail[j];  
8          if (s[j + 1] == s[i]) j++;  
9          fail[i] = j;  
10     }  
11 }
```

时间复杂度为 $O(n)$. 外层循环每次至多使 j 增加 1; 每次执行 $j = \text{fail}[j]$ 时 j 至少减少 1, 故 while 总共至多执行 $O(n)$ 次.

KMP 算法

字符串匹配：对于两个字符串 S 和 T . 在文本串 T 中寻找哪些子串与模式串 S 相同.

之前使用字符串哈希解决了该问题. 下介绍 KMP 算法解决字符串匹配问题.

KMP 算法

回顾之前的例子：考虑在文本串 $T = \text{ababababac}$ 中匹配模式串 $S = \text{abababac}$.

第一次对齐时，模式串的前 7 个字符均匹配成功，但最后一个字符失配：

T: ababababac

S: abababac

若接下来发生失配，朴素算法会将模式串向后移动一位，从头重新比较.

我们总是想要下一个 T 的字符满足匹配，我们能否不动当前 T 匹配到哪一个位置？去改变 S 开头的位置，让 T 的下一个字符满足匹配？

KMP 算法

我们总是想要下一个 T 的字符满足匹配，我们能否不动当前 T 匹配到哪一个位置？去改变 S 开头的位置，让 T 的下一个字符满足匹配？

对已经匹配的字符串 abababa，有

abababa abababa.
 └───┘ └───┘
 前缀 后缀

因此前一次匹配的末尾 ababa，可以直接作为下一次匹配的开
头；我们不必重新比较这 5 个字符。

T: ababababac
S: abababac

KMP 算法

对模式串 S 求得 $fail[1], fail[2], \dots, fail[|S|]$.

对于文本串 T 的某一位置 n , 记 j ($0 \leq j \leq \min\{|S|, n\}$) 为最大的整数满足 $S[1, j] = T[n - j + 1, n]$.

考虑对 $n = 1, 2, 3, \dots, |T|$, 求得满足上述条件的整数 j . 这显然比原问题更强.

KMP 算法

考虑递推. 若已经求得了 $n-1$ 的 j , 满足 $S[1, j] = T[n-j, n-1]$ 且 j 最大. 考虑如何得到 n 的 j (为避免记号重复, 下记 n 的 j 为 j'). 与之前得到 $fail$ 数组的方法类似.

我们按照同样的方式, 可以说明, j' 需要满足:

$S[1, j' - 1] = S[1, j]$, 或 $S[1, j' - 1]$ 是 $S[1, j]$ 的 border.

考虑从大到小枚举可行的 $j' - 1$ 值. 即枚举 $j, fail[j], fail[fail[j]], fail[fail[fail[j]]], \dots$, 直至枚举到 0 停止.

设枚举的数为 k . 检查 $S[k+1, n] = T[1, n-k]$ 是否成立. 若成立立刻停止枚举, 有最大的 $j' = k+1$ 满足 $S[1, j'] = T[n-j'+1, n]$. 若一直不成立, 则 $j' = 0$.

示例代码

```
1  int fail[MAX_N];
2  char s[MAX_N], t[MAX_N];
3  int n, m, ans[MAX_N], ans_size = 0;
4  void kmp() {
5      getmaxborder();
6      for (int i = 1, j = 0; i <= m; i++) {
7          while (j > 0 && s[j + 1] != t[i])
8              j = fail[j];
9          if (s[j + 1] == t[i]) j++;
10         if (j == n) {
11             ans[++ans_size] = i - n + 1;
12             j = fail[j];
13         }
14     }
15 }
```

时间复杂度为 $O(n + m)$. 分析方式与求 *fail* 数组的方式类似.

Border 树

Border 有良好的性质:

$$\text{border}(S) = \{\text{maxborder}(S)\} \cup \text{border}(\text{maxborder}(S))$$

考虑对每一个前缀 $S[1, i]$ ($0 \leq i \leq |S|$) 建立一个对应的结点 i . 每个结点 u ($u \neq 0$) 有父亲 $|\text{maxborder}(S[1, u])|$. 这样就建立了一颗树. 树的根为 0.

每个结点 u 的所有祖先的结点编号, 是 $S[1, u]$ 的所有 border 的长度, 且这些节点按照结点深度越深、结点编号越大、对应的 border 长度越长排列.

这样一颗树被称为 border 树. 也被称为 fail 树, 失配树.

Luogu P2375 失配树

给定一个串串 S ，询问两个前缀 $S[1, i]$ 和 $S[1, j]$ 的最长公共 border 的长度。

最长公共 border，即字符串 S' 同时满足 S' 是 $S[1, i]$ 的 border、是 $S[1, j]$ 的 border。

要求单次询问复杂度 $O(\log |S|)$ ，预处理复杂度 $O(|S| \log |S|)$ 。

Luogu P2375 失配树 Solution

给定一个串串 S , 询问两个前缀 $S[1, i]$ 和 $S[1, j]$ 的最长公共 border 的长度.

最长公共 border, 即字符串 S' 同时满足 S' 是 $S[1, i]$ 的 border、是 $S[1, j]$ 的 border.

建立 border 树. 每个结点 u 的所有祖先的结点编号, 是 $S[1, u]$ 的所有 border 的长度. 故 $S[1, i]$ 和 $S[1, j]$ 的公共 border 的长度, 是 i 和 j 的公共祖先.

同时, 结点深度越深、对应的 border 长度越长. 故 i 和 j 祖先中深度最深的点, 即 i 和 j 的最近公共祖先 (LCA) 的结点编号, 是最长公共 border 的长度.

使用倍增求 LCA. 询问复杂度 $O(\log |S|)$.

NOI2014 动物园

给定一个字符串 S . 对 S 的每个前缀 S' , 求长度小于等于 $\frac{|S'|}{2}$ 的 border 个数. 该题空串不计为 border.

要求时间复杂度为 $O(|S|)$ 或者 $O(|S| \log |S|)$.

NOI2014 动物园 Solution 1

给定一个字符串 S . 对 S 的每个前缀 S' , 求长度小于等于 $\frac{|S'|}{2}$ 的 border 个数. 该题空串不计为 border.

先不考虑长度限制 $\leq \frac{|S'|}{2}$. 求所有前缀 $S[1, i]$ 的 border 个数 $C[i]$.

根据 $\text{border}(S) = \{\text{maxborder}(S)\} \cup \text{border}(\text{maxborder}(S))$, 有递推:

$$C[i] = C[\text{Fail}[i]] + 1. \quad C[0] = C[1] = 0.$$

故可以 $O(|S|)$ 计算出 $C[i]$ ($0 \leq i \leq |S|$).

NOI2014 动物园 Solution 1

给定一个字符串 S . 对 S 的每个前缀 S' , 求长度小于等于 $\frac{|S'|}{2}$ 的 border 个数. 该题空串不计为 border.

下考虑长度限制 $\leq \frac{|S'|}{2}$. 设满足长度限制条件下前缀 $S[1, i]$ 的最长的 border 为 $conborder[i] = S[1, k]$ ($k \leq \frac{i}{2}$).

则所有满足条件的 border 为: $\{conborder\} \cup border(conborder)$.
故 $S[1, i]$ 满足条件的 border 个数为 $C[|conborder|] + 1$.

如何求出 $conborder[i]$ 的长度 k ?

若从大到小不断的枚举可能成立的 border 的长度 $fail[i]$,
 $fail[fail[i]]$, $fail[fail[fail[i]]]$, $\dots$, 判断是否满足条件. 并没有性质保证枚举 border 的数量级, 可以构造一个全部由同一个字符组成的字符串, 此时枚举的数量级为 $O(i)$, 总复杂度为 $O(|S|^2)$.

如何优化?

NOI2014 动物园 Solution 1

考虑这样一个性质：假设 $S[1, k]$ 是 $S[1, i]$ 的 border, $S[1, k - 1]$ 也是 $S[1, i - 1]$ 的 border. 若 $k - 1 > \frac{i-1}{2}$, 则 $k > \frac{i}{2}$.

考虑递推. 若我们已经求出来 $conborder[n - 1]$, 设其值为 x . 与求 fail 数组类似的过程, 枚举 $x, fail[x], fail[fail[x]], \dots$, 直至枚举到 0 停止 (根据上述性质, 比 x 大的 border, 一定不满足条件).

设枚举的数为 j . 检查 $S[j + 1] = S[n]$ 且 $j + 1 \leq \frac{n}{2}$ 是否成立. 若成立立刻停止枚举, 求出 $conborder[n] = j + 1$. 若一直不成立, $conborder[n] = 0$.

与求 fail 数组类似的方式分析复杂度. 复杂度为 $O(|S|)$.

NOI2014 动物园 Solution 2

给定一个字符串 S . 对 S 的每个前缀 S' , 求长度小于等于 $\frac{|S'|}{2}$ 的 border 个数. 该题空串不计为 border.

建立 border 树. 根据题意, 对一个结点 u , 其所有祖先的结点编号, 是 $S[1, u]$ 的所有 border 的长度. 故结点 u 的祖先中结点编号小于等于 $\frac{u}{2}$ 的结点个数 (需减去空串, 即根节点 0), 即为所要求的答案.

结点深度越深、结点编号越大. 故我们只要求出首个小于等于 $\frac{u}{2}$ 的结点, 其以上的结点均有贡献. 其深度即为答案 (若记跟结点深度为 0).

这一点使用倍增可以做到. 复杂度为 $O(|S| \log |S|)$. 该做法可能略微卡常.

① Formal Notation

② String Hash

③ KMP Algorithm

④ Border Theory

⑤ Trie

Border 理论

由于 border 理论的应用较为复杂，下只介绍 border 理论的主体内容，不做过多扩展.

下述内容可以更好的理解与分析 border 的结构与性质.

周期定义

设 $S = \text{abccabccab}$ ，它可以由长度为 3 的字符串 abc 不断重复，而形成 S 。我们称 S 有周期 3。

形式化地，若存在一个正整数 k ($1 \leq k \leq |S|$)，使得对于任意的 $1 \leq i \leq |S| - k$ 均有 $S[i] = S[i + k]$ ，则称 k 是 S 的一个周期。

若 $k \mid |S|$ ，则称该周期为整周期。整周期也可以被称为循环节。

对于一个字符串 S ，其最小的周期为 $\text{per}(S)$ 。

若 k 是周期，则不超过 $|S|$ 的 k 的正整数倍也是周期。

Border 与周期

定理 2

对于一个字符串 S ，下面两命题等价：

- S 有长度为 k 的 border.
- S 有长度为 $|S| - k$ 的周期.

Border 与周期

证明:

先证充分性. 若 S 有长度为 k 的 border, 则
 $S[1, k] = S[|S| - k + 1, |S|]$.

故 $S[1] = S[|S| - k + 1], S[2] = S[|S| - k + 2],$
 $S[3] = S[|S| - k + 3], \dots, S[k] = S[|S|]$.

即 $S[i] = S[|S| - k + i] \ (1 \leq i \leq k)$.

根据周期的定义, S 有长度为 $|S| - k$ 的周期. 得证.

Border 与周期证明

后证必要性. 若 S 有长度为 $|S| - k$ 的周期, 则

$$S[i] = S[|S| - k + i] \quad (1 \leq i \leq k).$$

$$\text{故 } S[1] = S[|S| - k + 1], S[2] = S[|S| - k + 2],$$

$$S[3] = S[|S| - k + 3], \dots, S[k] = S[|S|].$$

$$\text{故 } S[1]S[2] \cdots S[k] = S[|S| - k + 1]S[|S| - k + 2] \cdots S[|S|]. \text{ 故}$$

$$S[1, k] = S[|S| - k + 1, |S|].$$

S 有长度为 k 的 border. 得证.

Border 与周期推论

$$|\text{maxborder}(S)| = |S| - \text{per}(S).$$

POI2006 Periods of Words

给定字符串 S ，求 S 所有前缀的最大的周期。

要求时间复杂度： $O(|S|)$ 。

弱周期引理 (WPL)

英文名 Weak Periodicity Lemma (WPL).

定理 3

若 p, q 是 S 的周期, 且 $p + q \leq |S|$, 则 $\gcd(p, q)$ 也是 S 的周期.

弱周期引理 (WPL) 证明

$p = q$ 时显然成立.

不失一般性的, 令 $p > q$. 考虑证明 $p - q$ 也是周期.

而后我们对 $p - q, q$ 这两个周期重复上述过程. 若我们不断重复, 则两数的 gcd 保持不变并最后两数相等时等于 $\gcd(p, q)$. 这类似于求 gcd 的辗转相除法, 将取模运算改成不断相减是等价的.

q 一定满足 $q \leq \frac{|S|}{2}$.

当 $1 \leq i \leq p$ 时, $S[i] = S[i + p] = S[(i + p) - q] = S[i + (p - q)]$.

当 $p < i \leq |S| - (p - q)$ 时,
 $S[i] = S[i - q] = S[(i - q) + p] = S[i + (p - q)]$.

故 $p - q$ 是 S 的一个周期.

SNPCPC2026 J 世末歌者

该题依赖一个性质：

给定一个仅由字符 0 和 1 构成的字符串 S ，且具有整周期 $k \neq |S|$.

对该字符串任意位置 01 取反，得到一个新的字符串 S' . S' 没有任何的除 $k = |S|$ 外的整周期.

尝试证明？

周期引理 (PL)

英文名 Periodicity Lemma (PL).

定理 4

若 p, q 是 S 的周期, 且 $p + q - \gcd(p, q) \leq |S|$, 则 $\gcd(p, q)$ 也是 S 的周期.

不证. 通常 WPL 已经足够强.

Border 的结构

定理 5

S 的长度大于等于 $\frac{|S|}{2}$ 的 border 的长度构成一个等差数列.

Border 的结构证明

若不存在长度大于等于 $\frac{|S|}{2}$ 的 border, 则结论显然正确.

设 $|\text{maxborder}(S)| = |S| - p$. 设有另一个 border 长度为 $|S| - q$ 且 $|S| - q \geq \frac{|S|}{2}$. 则 p 和 q 是 S 的周期. 且 $\text{per}(S) = p$.

同时 $|S| - p \geq |S| - q \geq \frac{|S|}{2}$, 可得 $p, q \leq \frac{|S|}{2}$. 由 WPL 知 $\text{gcd}(p, q)$ 也是 S 的周期.

根据 $\text{per}(S) = p$, 即 p 是最小的周期, 可知 $p \leq \text{gcd}(p, q)$. 故 $p \mid q$.

同时, 存在周期 kp ($1 \leq k \leq \frac{|S|}{p}$). 这意味着, 小于等于 $\frac{|S|}{2}$ 的周期, 一定是 p 的整数倍且 p 的整数倍一定是周期.

border 的长度与 $|S|$ 减周期一一对应. 即长度大于等于 $\frac{|S|}{2}$ 的 border 构成等差数列, 公差为 p .

Border 的结构

定理 6

S 的所有 border 长度可以被拆分成 $O(\log |S|)$ 个等差数列.

更严的上界, $\lceil \log_2 |S| \rceil$.

Border 的结构证明

考虑将 S 的 border 按照长度 $[0, 2), [2, 4), [4, 8), [8, 16), \dots, [2^k, |S|)$, 分成 $\lceil \log_2 |S| \rceil$ 个集合.

在 border 长度为 $[0, 2)$ 集合中, 至多存在 2 个 border, 构成等差数列.

对于 border 长度为 $[2^k, \min\{2^{k+1}, |S|\})$ 的集合, 可以找到一个最长的 border, 记为 T .

根据 $\text{border}(S) = \{\text{maxborder}(S)\} \cup \text{border}(\text{maxborder}(S))$, 可知 S 的 border 长度小于 $\min\{2^{k+1}, |S|\}$ 的集合, 为 $T \cup \text{border}(T)$.

根据定理 5, 长度大于等于 $\frac{|T|}{2}$ 的 border 的长度构成一个等差数列. 而 $\frac{|T|}{2} \leq \frac{2^{k+1}}{2} = 2^k$, 故长度为 $[\frac{|T|}{2}, \min\{2^{k+1}, |S|\}) \subseteq [2^k, \min\{2^{k+1}, |S|\})$ 构成的 border 集合, 其长度为等差数列.

① Formal Notation

② String Hash

③ KMP Algorithm

④ Border Theory

⑤ Trie

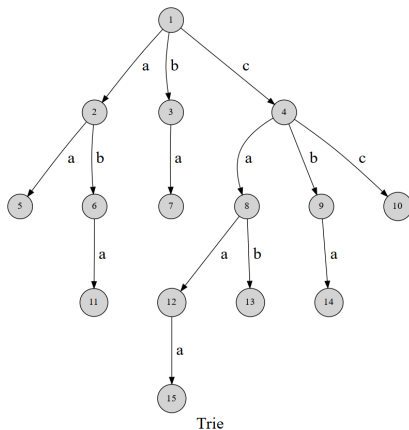
Trie 简介

trie 树, 或者称字典树.

设给定一个字符串集合.

trie 树通常维护字符串集合的操作. 例如检索一个字符串 S 是否在该字符串集合中.

Trie



用边来代表字母. 从根结点到树上某一结点的路径就代表了一个字符串. 例如, $1 \rightarrow 4 \rightarrow 8 \rightarrow 12$ 表示的就是字符串 `caa`.

示例代码

代码中以 0 为根. 设字符集为所有小写字母.

```
1  int nxt[MAX_N][26], cnt = 0;
2  bool exist[MAX_N];
3  void insert(char *s, int len) {
4      int p = 0;
5      for (int i = 0; i < len; i++) {
6          int c = s[i] - 'a';
7          if (!nxt[p][c]) {
8              nxt[p][c] = ++cnt;
9          }
10         p = nxt[p][c];
11     }
12     exist[p] = 1;
13 }
```

示例代码

```
1  bool find(char *s, int len) {
2      int p = 0;
3      for (int i = 0; i < len; i++) {
4          int c = s[i] - 'a';
5          if (!nxt[p][c]) {
6              return 0;
7          }
8          p = nxt[p][c];
9      }
10     return exist[p];
11 }
```

Trie

设插入的字符串为 $S_1, S_2, S_3, \dots, S_n$. 字符集为 Σ .

时间复杂度:

- 单次插入字符串 S_i : $O(|S_i|)$.
- 单次询问字符串 S 是否在集合中: $O(|S|)$.

空间复杂度: $O(|\Sigma| \sum_{i=1}^n |S_i|)$.

01 Trie

01 字符串指字符集为 $\{0, 1\}$ 的字符串.

01 Trie 指字符集为 $\{0, 1\}$ 的 trie 树.

通常在若干个非负整数 $a_1, a_2, a_3, \dots, a_n$ 中, 维护位运算相关操作.

我们可以将 a_i 转换成二进制数, 以此作为 01 字符串 S , 插入 trie 树中.

有时, 字符串 S 的 $S[1]$ 为二进制数最高位, 有时为最低位. 若 $S[1]$ 为最高位, 则通常 01 字符串的长度需取一极大值. 若某一二进制数不足, 则高位补前导 0.

维护的具体问题应具体分析. 下介绍一个经典的例子.

01 Trie

给定若干个非负整数 $a_1, a_2, a_3, \dots, a_n$. 求 $a_i \oplus a_j$ 的最大值 ($1 \leq i, j \leq n$).

设 a_i 的最大值为 V . $V < 2^{31}$.

要求时间复杂度 $O(n \log V)$.

01 Trie

给定若干个非负整数 $a_1, a_2, a_3, \dots, a_n$. 求 $a_i \oplus a_j$ 的最大值 ($1 \leq i, j \leq n$).

设 a_i 的最大值为 V . $V < 2^{31}$.

将 a_i 转换成二进制数, 二进制数不足 31 位在高位补前导 0, 以此作为 01 字符串 S_i . 以 $S_i[1]$ 作为二进制数最高位, 将 n 个 01 字符串插入 trie 树中.

考虑枚举 a_i . 对每个 a_i , 求 $a_i \oplus a_j$ 的最大值. 从 Trie 的根开始, 贪心的, 如果能向 a_i 当前位不同的子树走, 则往该子树走 (即存在一个 a_j , 当前位与 a_i 不同). 否则, 往另一方向走.

如果这么走, 则这一位为 1. 否则为 0. 而越开始的位越高, 高位需优先选择大的, 肯定更优.

Thanks!